DSC 140A - Homework 02

Homework 02 Instructions. You may collaborate with whomever, including AI. Each student must submit their own homework even if they worked with others. We will not be grading for correctness, but we will be grading for completeness. Please turn out a single PDF file that contains your solutions to all of the problems. We recommend using LaTeXwith our template. You can use markdown but make sure each problem is clearly labeled. For coding problem, we recommend using Jupyter notebooks, covert them into PDF, and then merge the PDF with your math problems solutions generated in LaTeX.

Problem 1.

The following is a common question you might see in an interview for a machine learning job. The question is: Should you scale your features before performing least squares?

Your friend thinks that scaling the features before performing least squares regression is a good idea. Their logic is that scaling sometimes improves the performance of k-nearest neighbors predictors, so it might help with least squares as well. But you're not so sure.

In this problem, we'll show that scaling the features before performing least squares regression does not actually change the predictions that are made, and so it doesn't improve the performance of the model[1].

Hint: It might be helpful to use some of the matrix-vector algebra properties covered in the math check. If you use a property that wasn't listed in the math check or lecture, it's OK, but make sure to explicitly state the property you're using.

a) Let $(\vec{x}^{(1)}, y_1), (\vec{x}^{(2)}, y_2), \ldots, (\vec{x}^{(n)}, y_n)$ be a training set of n feature vectors in $\mathbb{R}^d$ and their corresponding labels. Recall that the decision matrix X is

$$X = \begin{pmatrix} 1 & \vec{x}_1^{(1)} & \vec{x}_2^{(1)} & \cdots & \vec{x}_d^{(1)} \\ 1 & \vec{x}_1^{(2)} & \vec{x}_2^{(2)} & \cdots & \vec{x}_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \vec{x}_1^{(n)} & \vec{x}_2^{(n)} & \cdots & \vec{x}_d^{(n)} \end{pmatrix}$$

Suppose we will scale feature i by a constant factor of c_i in each feature vector. The result is a scaled data set whose design matrix X_C is:

$$X_C = \begin{pmatrix} 1 & c_1\vec{x}_1^{(1)} & c_2\vec{x}_2^{(1)} & \cdots & c_d\vec{x}_d^{(1)} \\ 1 & c_1\vec{x}_1^{(2)} & c_2\vec{x}_2^{(2)} & \cdots & c_d\vec{x}_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & c_1\vec{x}_1^{(n)} & c_2\vec{x}_2^{(n)} & \cdots & c_d\vec{x}_d^{(n)} \end{pmatrix}$$

Find a diagonal matrix C so that $X_C = XC$.

b) In lecture, we will see that for the unscaled data, the parameter vector that minimizes the mean squared error is given by $\vec{w}^* = (X^TX)^{-1}X^T\vec{y}$. When we scale the features, however, the optimal weight vector becomes $\vec{w}_C^* = (X_C^TX_C)^{-1}X_C^T\vec{y}$.
Show that $\vec{w}_C^* = C^{-1}\vec{w}^*$ by using the fact that $X_C = XC$.
Hint: $(AB)^{-1} = B^{-1}A^{-1}$ (as long as A and B are invertible).

[1] This is not to say that scaling is never useful when doing least squares regression. Scaling the features can make the weights easier to interpret, for example, or help with numerical stability.

c) Consider a new point $\vec{x}$ for which we want to make a prediction.

The prediction made by the original model is $\vec{w}^* \cdot \text{Aug}(\vec{x})$.

The prediction made by the new model trained on scaled data is $\vec{w}_C^* \cdot \text{Aug}(\vec{x}_C)$, where $\vec{x}_C$ is $\vec{x}$ scaled by the same factors as the training data. It can be shown that $\text{Aug}(\vec{x}_C) = C\,\text{Aug}(\vec{x})$.

Using these facts and the result of the previous parts, show that $\vec{w}^* \cdot \text{Aug}(\vec{x}) = \vec{w}_C^* \cdot \text{Aug}(\vec{x}_C)$; that is, both models make exactly the same prediction, and scaling had no effect.

Hint: The inverse of a diagonal matrix is also diagonal. The transpose of a diagonal matrix is the same as the original matrix.

Problem 2.

Let $f(\vec{z}) = e^{z_1} + e^{z_2} + e^{z_3} + (z_1 - 1)^2 + \|\vec{z}\|^2$ be a function of $\vec{z} = (z_1, z_2, z_3)^T$.

Find a minimizer of this function using gradient descent (implemented with code; don't run GD by hand!). Report:

- The initial point, $\vec{z}^{(0)}$;
- your choice of step size;
- the stopping criterion you chose and number of iterations taken to reach the criterion;
- the minimizer you found;
- the minimum value of the function.

Include your code.

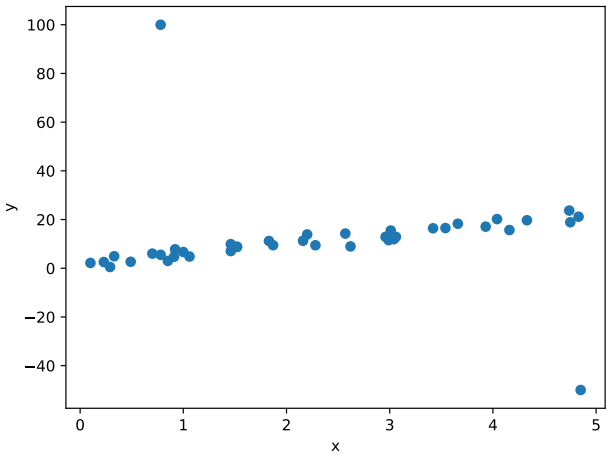
Problem 3.

The file at the link below contains a simple data set suitable for regression.

`https://f000.backblazeb2.com/file/jeldridge-data/002-regression_outlier/data.csv`

The first column contains the x values (the predictor variable) and the second column contais the y values (the target).

The plot below shows the data:



Notably, the data contains outliers which may affect our regression.

Fit a linear function of the form $w_0 + w_1 x$ by minimizing the mean squared error. Report w_0 and w_1, plot the fitted line, and include your code.

Do not use `sklearn`, `scipy`, or any library except for `numpy` and `matplotlib` for this problem. You may use the code given in lecture for least squares regression.

Problem 4.

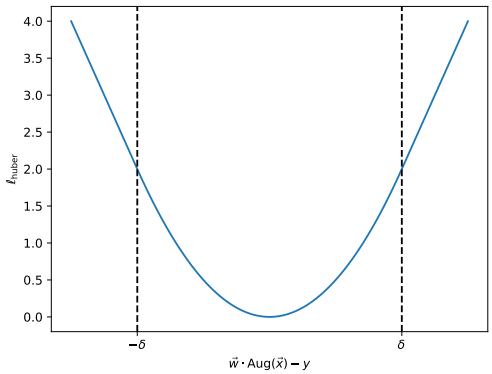
In the previous problem, you performed least squares regression on a data set.

As you can see, the data contains outliers which may affect our regression. In the previous homework, you should have found that least squares regression is not robust to these outliers.

The *Huber loss* is a loss function used in regression that is less sensitive to outliers than the square loss. It can be thought of as a "mix" between the square loss and the absolute loss. For linear prediction functions $H(\vec{x}) = \vec{w} \cdot \text{Aug}(\vec{x})$, the Huber loss is defined as:

$$\ell_{\text{huber}}(\text{Aug}(x) \cdot \vec{w}, y) = \begin{cases} \frac{1}{2}(\text{Aug}(x) \cdot \vec{w} - y)^2, & \text{if } |\text{Aug}(x) \cdot \vec{w} - y| \leq \delta, \\ \delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2, & \text{if } \text{Aug}(x) \cdot \vec{w} - y > \delta, \\ -\delta(\text{Aug}(x) \cdot \vec{w} - y) - \frac{1}{2}\delta^2, & \text{if } \text{Aug}(x) \cdot \vec{w} - y < -\delta, \end{cases}$$

A plot of the Huber loss is shown below.



Note that, despite being a piecewise function, the Huber loss is differentiable (unlike the absolute loss). However, there is no closed-form solution for the minimizer of the empirical risk with respect to the Huber loss, so if we want to train a regression model using this loss we need to use an iterative optimization algorithm, like gradient descent.

a) Write a function to compute the risk of a linear prediction function $w_0 + w_1 x$ with respect to the Huber loss (with $\delta = 1$) for the data given above. Use your function to compute the risk for $\vec{w} = (20, -1)$. Report the risk, and include your code.

Self-check: By running `huber_risk([20, -1])`, we find that the risk of the linear prediction function $20 - x$ with respect to the Huber loss is approximately 11.085.

b) The gradient of the Huber loss with respect to $\vec{w}$ is also a piecewise function. Compute this gradient. Note that since the Huber loss is differentiable, the gradient can be computed by separately computing the gradient within each piece of the piecewise function. You may use any of the matrix-vector calculus rules we've seen in class to help you compute the gradient. For example, you may use the fact that $\frac{d}{d\vec{w}} \vec{w} \cdot \text{Aug}(\vec{x}) = \text{Aug}(\vec{x})$.

c) Write a function that computes the gradient of the empirical risk with respect to the Huber loss with $\delta = 1$ for a linear prediction function $w_0 + w_1 x$. Use your function to compute the gradient for $\vec{w} = (20, -1)$. Show your code.

d) Implement and run gradient descent to minimize the empirical risk with respect to the Huber loss (using $\delta = 1$) on the data set above. In addition to your code, include:

- The optimal solution found by gradient descent;
- A plot of the empirical risk of $\vec{w}^{(t)}$ at each iteration. In other words, include a plot whose horizontal axis is iteration number, t, and whose vertical axis measures the empirical risk of $\vec{w}^{(t)}$.

- A plot of the data and the linear prediction function corresponding to the optimal solution.

e) Implement and run *stochastic* gradient descent with a batch size of 8 to minimize the empirical risk with respect to the Huber loss (using $\delta = 1$) on the data set above.

Include:

- The optimal solution found by SGD;
- A plot of the empirical risk of $\vec{w}^{(t)}$ at each iteration. In other words, include a plot whose horizontal axis is iteration number, t, and whose vertical axis measures the empirical risk of $\vec{w}^{(t)}$.

Note that you should plot the risk with respect to the whole data set, and not the risk with respect to the random batch (the latter will be very noisy, and it's the former that we're trying to optimize).